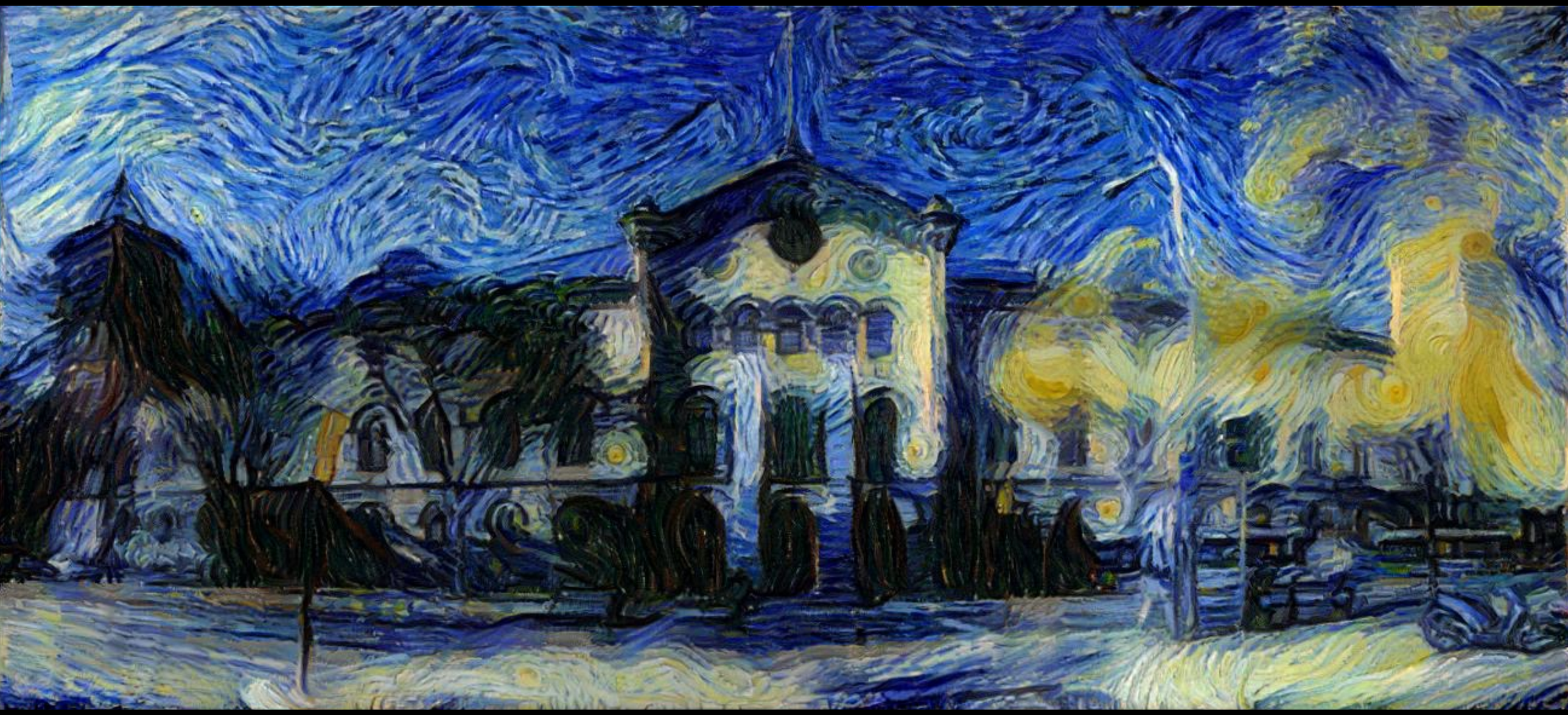
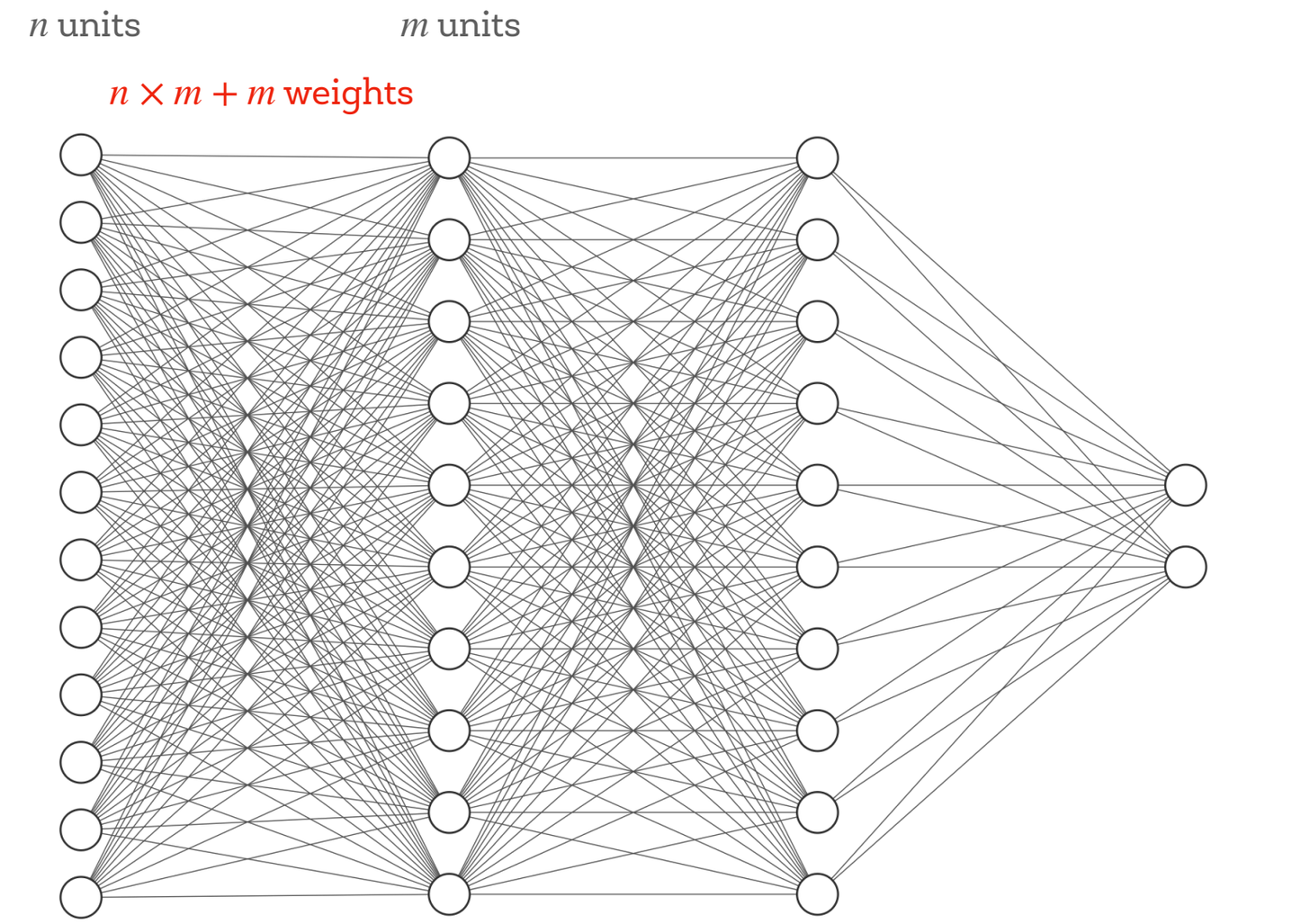


Deep Learning

Convolutional Neural Networks

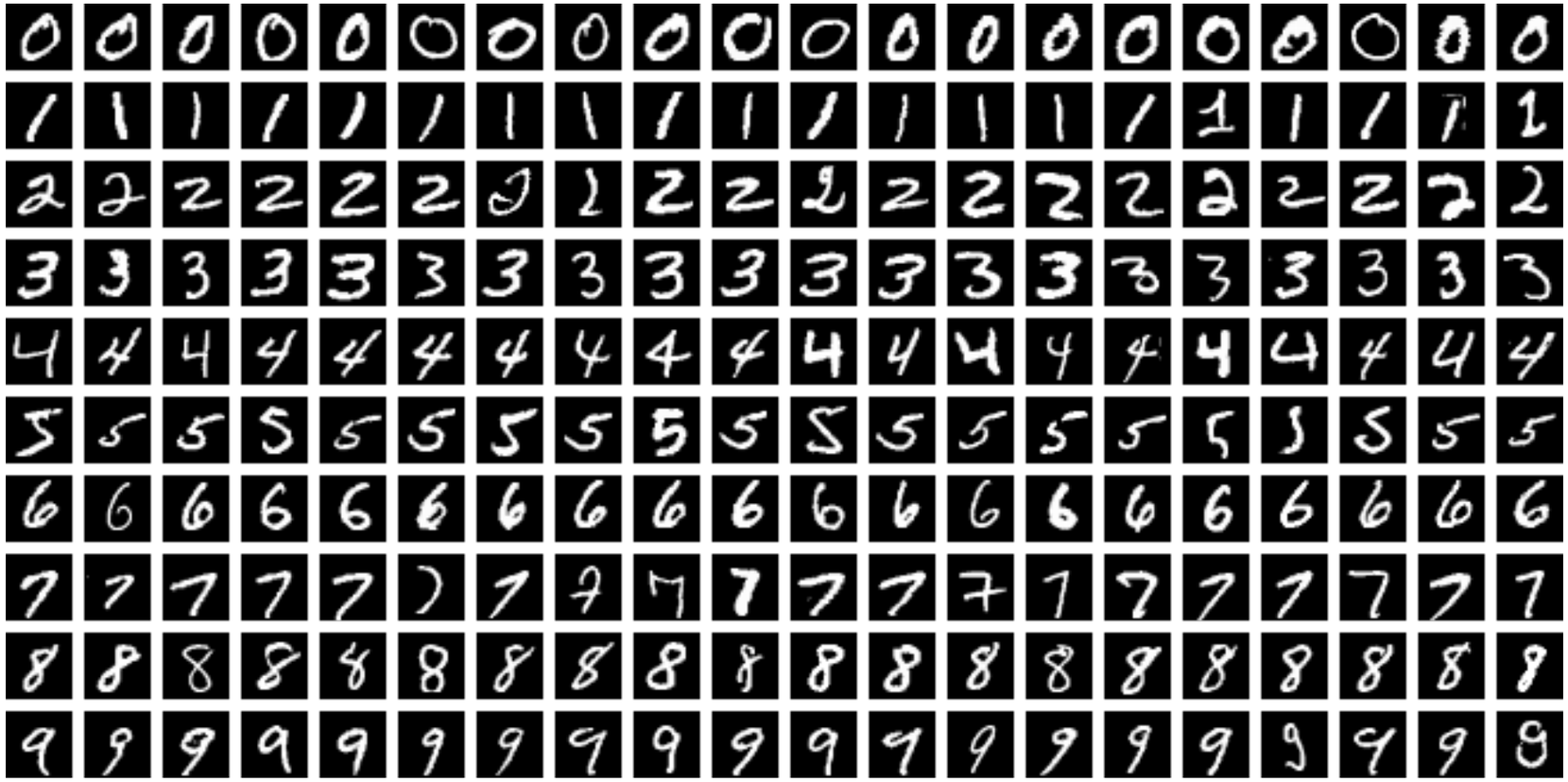
Images and FC neural networks



$12 \times 10 + 10 = 130$ weights $\quad 10 \times 10 + 10 = 110$ weights $\quad 10 \times 2 + 2 = 22$ weights

https://alexlenail.me/NN-SVG/index.html

Images and FC neural networks



MNIST (Modified National Institute of Standards and Technology) is a classic machine-learning benchmark of 70,000 grayscale images of handwritten digits from 0 to 9. It is split into **60,000 training images and 10,000 test images**.

Each image is 28×28 pixels, so 784 features per image, with values from 0 (background) to 255 (ink). The digits were size-normalized and centered, so they need almost no preprocessing.

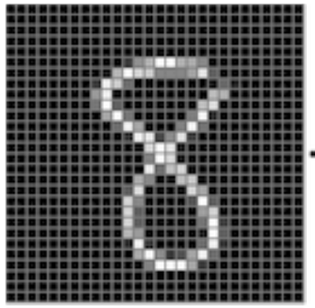
Each image's label is an integer from 0 to 9, and the ten classes are roughly balanced. Its small size, clean format and easy loading (keras.datasets.mnist.load_data()) make it the "hello world" of deep learning.

Images and FC neural networks

Original RGB image

↓

BW image



28 x 28

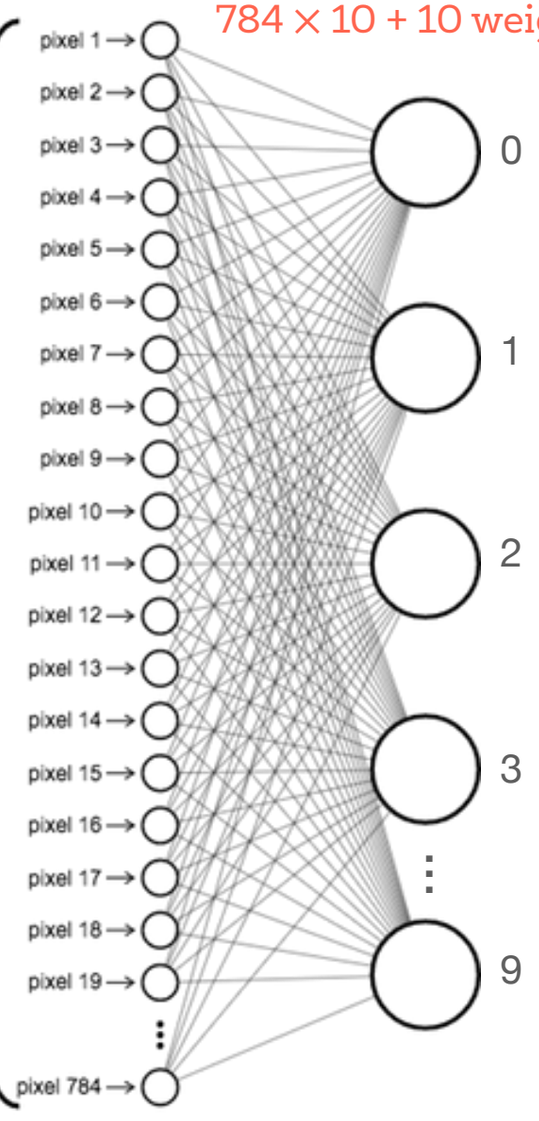
784 pixels

→

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	4	62	146	182	254	254	181	176	139	15	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	34	186	253	217	208	136	136	136	166	232	99	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	61	242	208	111	3	0	0	0	0	0	18	32	107	43	0	0	0	0	0	0
0	0	0	0	0	0	0	0	156	242	23	0	0	0	0	0	0	0	13	191	181	6	0	0	0	0	0	0
0	0	0	0	0	0	0	0	121	255	98	3	0	0	0	0	0	8	194	225	12	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	169	253	120	3	0	0	0	0	128	247	51	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	3	111	244	169	19	0	14	131	249	117	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	59	241	235	72	142	229	66	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	25	218	254	231	36	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	133	253	221	33	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	19	237	111	196	217	19	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	174	138	0	23	193	204	18	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	96	224	0	0	0	25	218	169	3	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	215	138	0	0	0	0	86	253	99	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	215	97	0	0	0	0	3	162	214	11	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	215	97	0	0	0	0	0	118	253	68	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	185	157	0	0	0	0	0	40	254	98	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	50	244	61	0	0	0	0	112	244	58	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	174	251	142	59	83	167	244	111	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	6	133	253	253	253	169	61	3	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Digit Recognition Problem

784 × 10 + 10 weights



Flattening operation.

This function is used to get a copy of a given tensor collapsed into one dimension.

What about processing a 1024x1024 image?

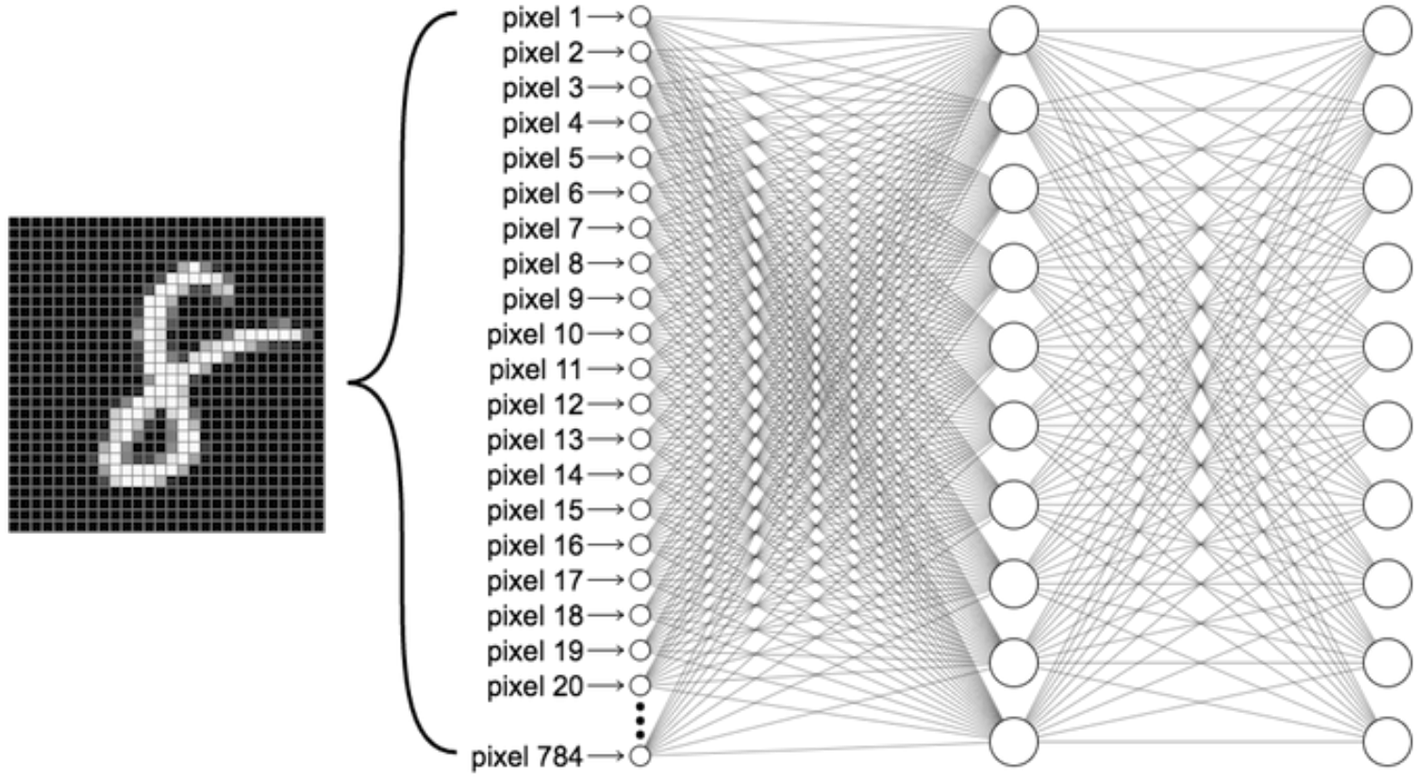
1024 × 1024 × 10 + 10 weights

https://mc.ai/what-is-deep-learning/

Part I: CNN layers

Images and FC neural networks

784 → 10 is a very aggressive dimensionality reduction!

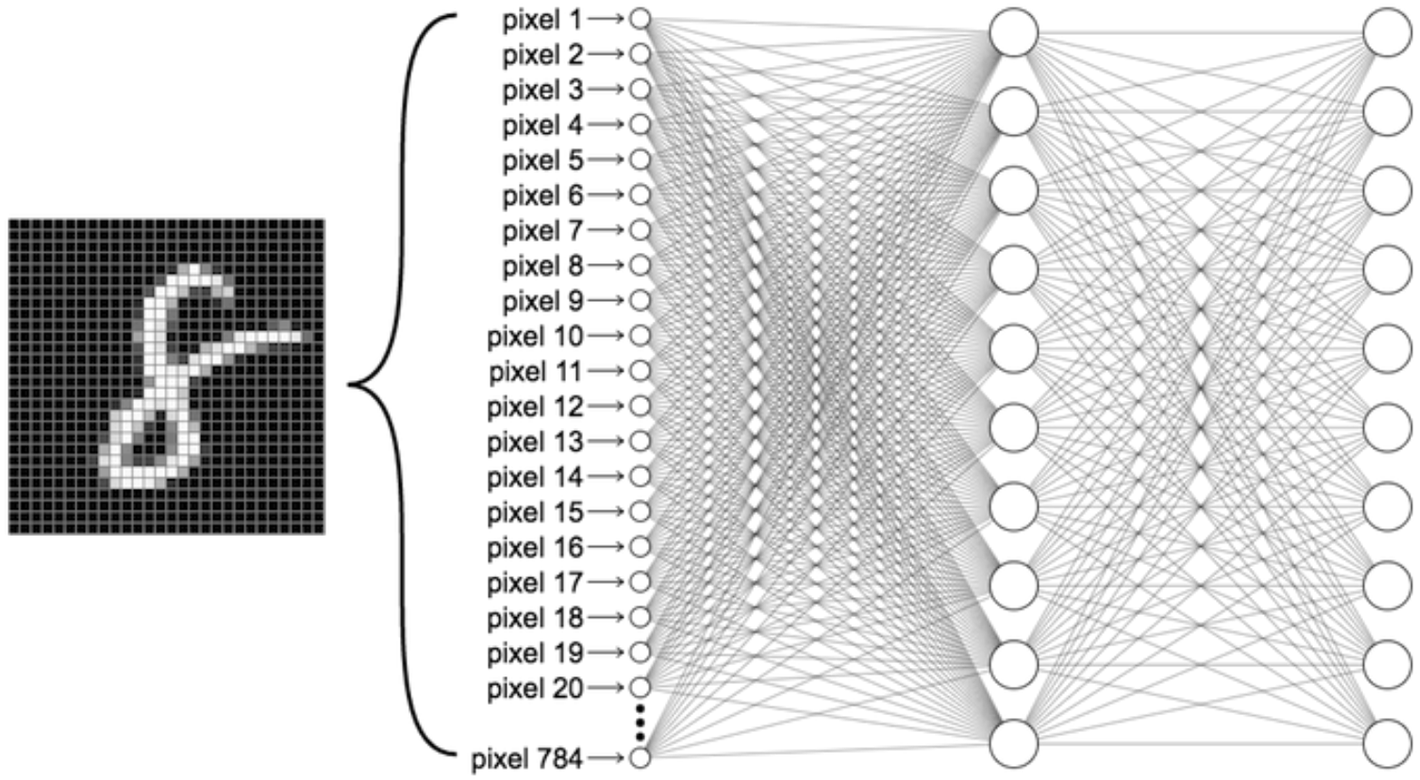


https://www.google.com/url?sa=i&url=https%3A%2F%2Fml4a.github.io%2Fml4a%2Flooking_inside_neural_nets%2F&psig=AOvVaw0UknUai0JapuIndBrY_BpX&ust=1604588386364000&source=images&cd=vfe&ved=0CAIQjRxqFwoTCOiB-NmT6ewCFQAAAAAdAAAAABAD

Say we have a dataset of one-megapixel photographs we need to classify.
This means that each input to the network has **one million dimensions**.

Even an **aggressive reduction** to 1000 hidden dimensions would require a fully-connected layer characterized by $10^6 \times 10^3 = 10^9$ **parameters**.
Learning a classifier by fitting so many parameters might require collecting an **enormous dataset**.

Images and FC neural networks



https://www.google.com/url?sa=i&url=https%3A%2F%2Fml4a.github.io%2Fml4a%2Flooking_inside_neural_nets%2F&psig=AOvVaw0UknUai0JapuIndBrY_BpX&ust=1604588386364000&source=images&cd=vfe&ved=0CAIQjRxqFwoTCOiB-NmT6ewCFQAAAAAdAAAAABAD

Convolutional neural networks (CNNs) are one creative way that machine learning has embraced for *exploiting some of the known structure in natural images* in order to (i) **minimize the number of parameters** and also (ii) to build interesting **invariances/inductive biases in the model**.

Inductive Bias

We can study **models** from two aspects:

- Whether a solution is **realizable** for the model, i.e., there is at least one set of weights that leads to the "optimal" solution.
- Whether a solution is **learnable** for the model, i.e., it is possible for the model to learn that solution within a reasonable amount of time and computational resources.

Inductive Bias

In many cases in deep learning, we are dealing with models that have enough (potential) expressive power to solve our problems.

However, they have **different biases regarding learnability**.

Optimal solutions are realizable for all of them, but depending on the task at hand it is easier for some of them to learn the solutions compared to the others.

Inductive Bias

Neural networks are good function approximation methods for **some kinds of data** (images, texts, etc.) because they can **leverage interesting invariances that are embedded in their architectures**.

This property is called **inductive bias**.

Inductive bias describes the tendency for a system to **prefer a certain set of generalizations over others** that are equally consistent with the observed data.

Inductive biases are the characteristics of learning algorithms that influence their generalization behavior, **independent of data**.

Inductive Bias

Suppose an unbiased algorithm is trying to learn a mapping from numbers in the range one to 100 to the labels TRUE and FALSE.

> It observes that “2”, “4” and “6” are labeled TRUE, while “1”, “3” and “5” are labeled FALSE.

What is the label for “7”?

> To the **unbiased learner**, there are 2^{94} possible labelings (each one implemented by a different classifier) of the numbers seven through 100, all equally possible, with “7” labeled TRUE in one half and “7” labeled FALSE in the other.

Inductive Bias

X	Y
1	0
2	1
3	0
4	1
5	0
6	1
7	?
8	?
9	?
10	?
...	...
99	?

$y = f(x)$

The X space has 100 elements. We have fixed the outcome for 6 elements $(x_1, x_2, x_3, x_4, x_5, x_6)$, and we want to learn a function to predict the outcome for the rest.

In principle, all these classifiers ($n = 2^{94}$) are equally possible!
Each one of them represents **a function that is compatible with the training data**.

Which one is the right one?

1	0
2	1
3	0
4	1
5	0
6	1
7	1
8	1
9	1
10	1
...	...
99	1

$\hat{y} = f_1(x)$

1	0
2	1
3	0
4	1
5	0
6	1
7	0
8	1
9	1
10	1
...	...
99	1

$\hat{y} = f_2(x)$

1	0
2	1
3	0
4	1
5	0
6	1
7	0
8	0
9	1
10	1
...	...
99	1

$\hat{y} = f_3(x)$

. . .

1	0
2	1
3	0
4	1
5	0
6	1
7	0
8	0
9	0
10	0
...	...
99	0

$\hat{y} = f_n(x)$

Inductive Bias

To a human, of course, there is a clear pattern of even numbers labeled TRUE and odds labeled FALSE, so a label of FALSE for "7" seems likely.

This type of **belief**, that certain hypotheses like

"*evens are labeled TRUE*"

seem likely, and other hypotheses like

"*evens are labeled TRUE up through "6", and the rest of the labeling is random*"

seem unlikely. This is **inductive bias**.

Inductive Bias

This likely solution can be seen as a "**simple solution**" from different points of view.

For example:

- It can be written in a concise way
  (if $x \,\%\, 2 = 0$: label $= 1$; else label $= 0$).
- Most of the 2^{94} solutions cannot be written in a way simpler than enumerating the outcome for each input value.

"**Simplicity**" (**parsimony**) **is one strong inductive bias**!

Inductive Bias

Fully-connected neural networks have an inherent inductive bias that makes them inclined to learn **generalizable hypotheses** and **avoid memorization**.

> In this respect, some results suggest (this is an hypothesis) that the inductive bias stems from neural networks being lazy: they tend to learn **simple rules** first.

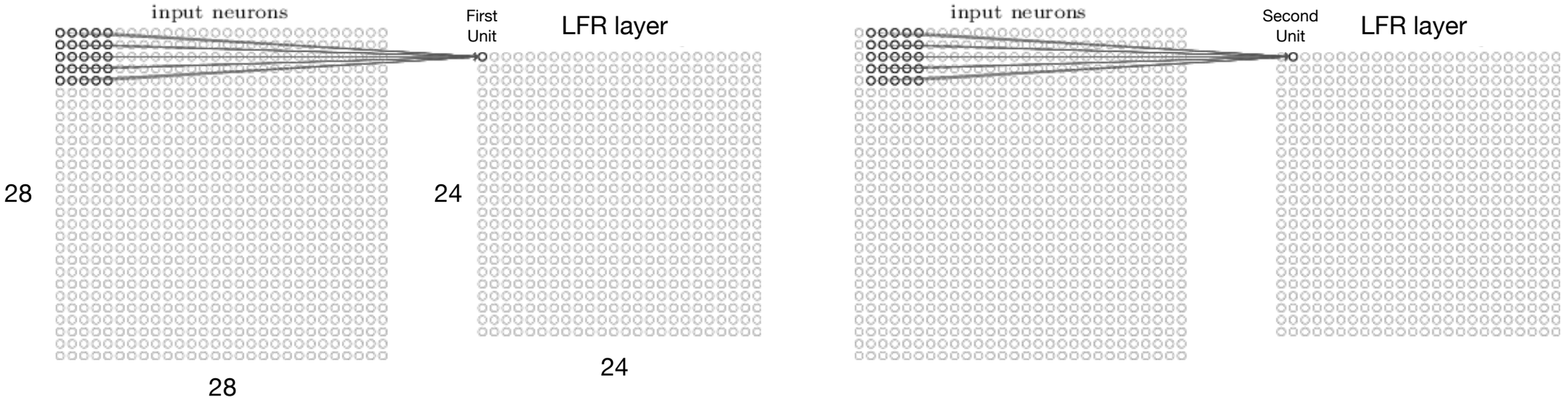
Moreover, deep learning **architectures** (beyond fully connected layers) add new inductive biases to these models.

Minimizing the number of parameters: LRF's

One possible to minimize the number of parameters is to consider **local receptive fields** (LRF) (an inspiration that comes from neuroscience).

A local receptive field is a standard MLP with **units that have a local view** of the image: its input is a submatrix $M \times M$ of the original $N \times N$, where $M \ll N$.

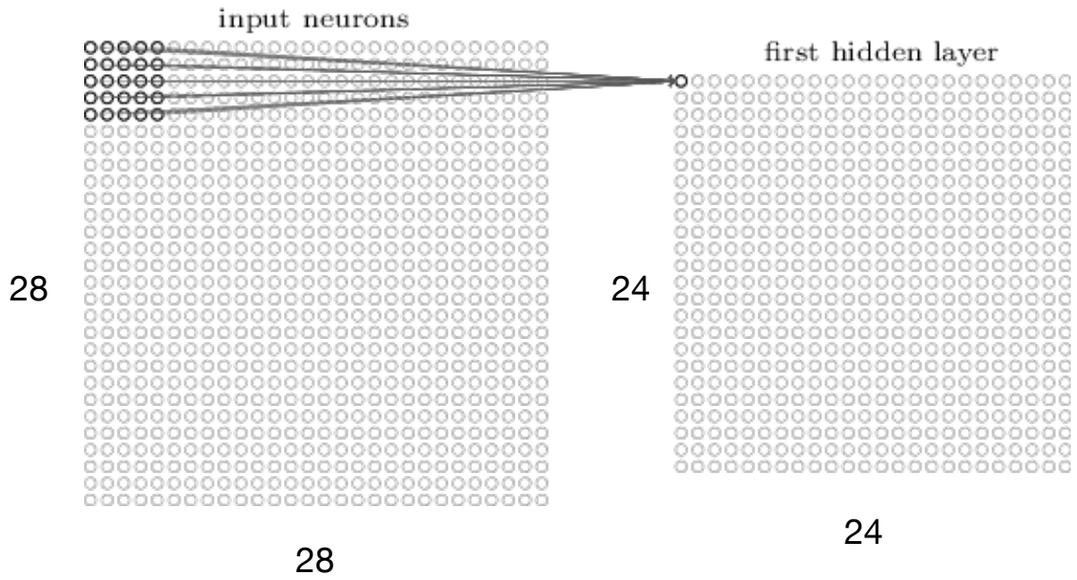
There is one bias for each output unit.



No flattening operation. Our layers are tensors.

http://neuralnetworksanddeeplearning.com/chap6.html

Minimizing the number of parameters: LRF's



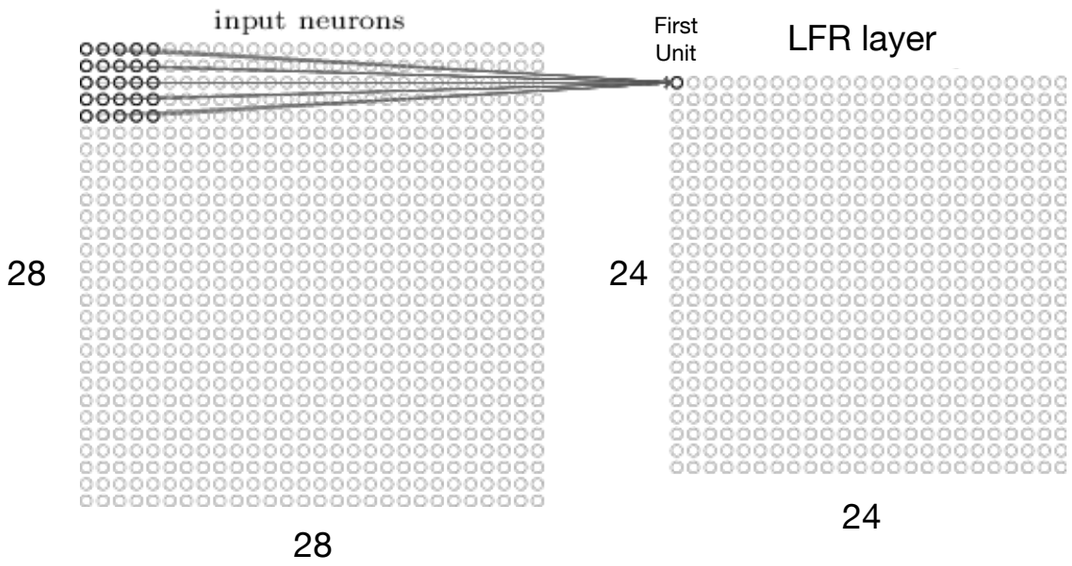
This LFR layer has one neuron for each pixel (with the exception of the borders of the image)

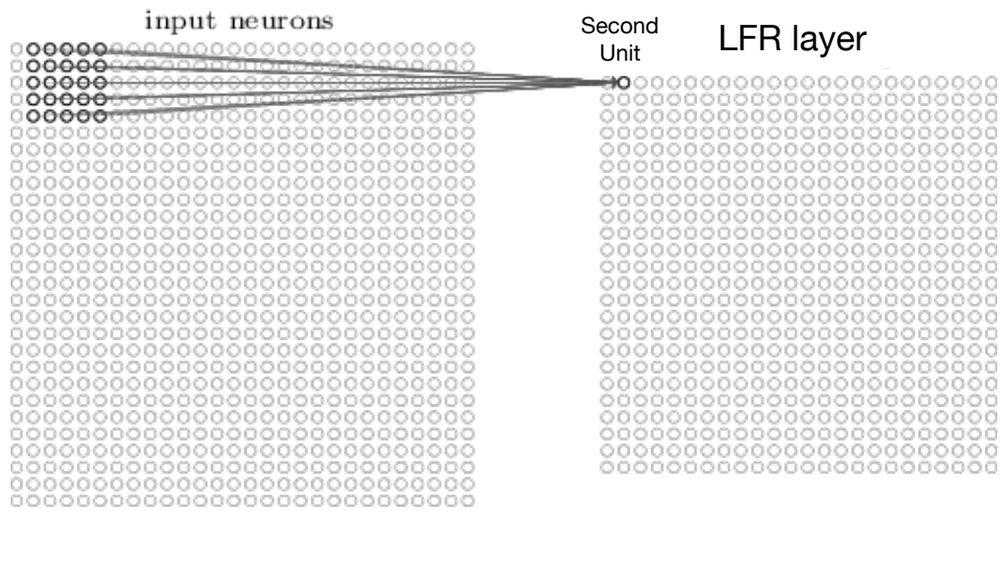
Note that if we have a 28×28 input image, and 5×5 local receptive fields, then there will be 24×24 neurons in the hidden layer.

This is because we can only move the local receptive field 24 neurons across (or 24 neurons down), before colliding with the right-hand side (or bottom) of the input image.

http://neuralnetworksanddeeplearning.com/chap6.html

Minimizing the number of parameters: LRF's





MNIST image size= 28 x 28 pixels = 784 dimensions

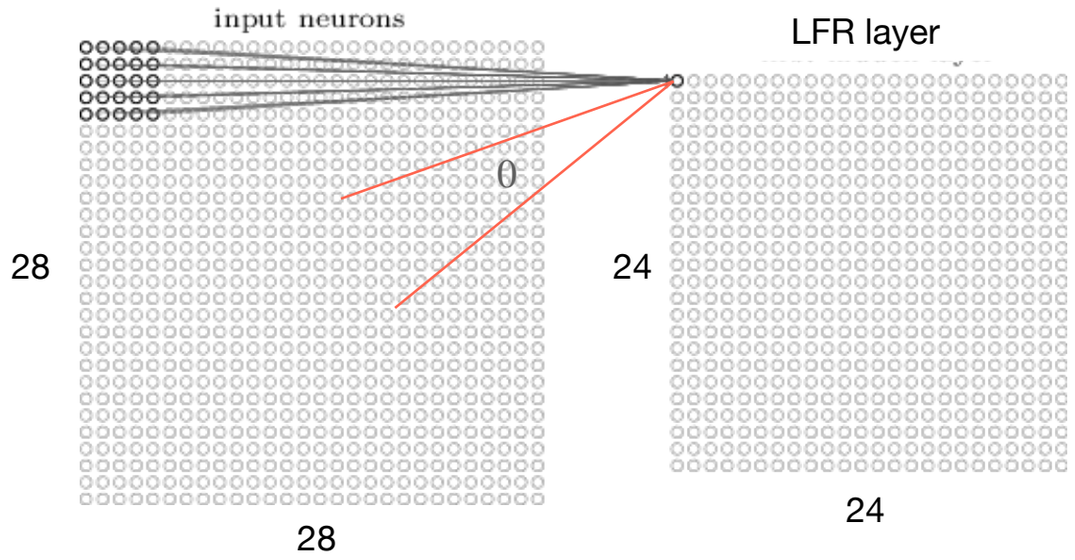
A FC layer with 100 units has (784 x 100 + 100) parameters = 78500 parameters

This LRF layer with 24x24 units has 25 x 24 x 24 + (24 x 24) parameters = 14976 parameters

http://neuralnetworksanddeeplearning.com/chap6.html

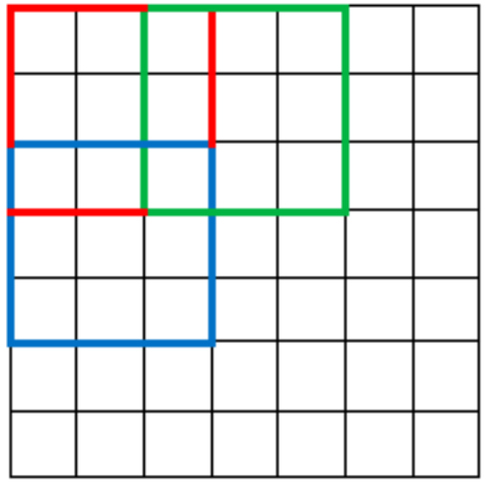
Minimizing the number of parameters: LRF's

A LRF network can be seen as a fully connected layer where a large subset (unique to each pixel location) of weights are 0.

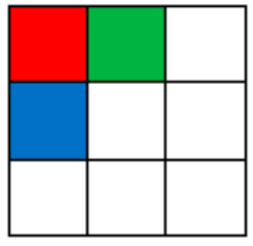


Minimizing the number of parameters: LRF's

Stride = 2



Input 7x7



Output 3x3

We have shown the local receptive field being moved by one pixel at a time, but this can be **redundant** in the case of natural images.

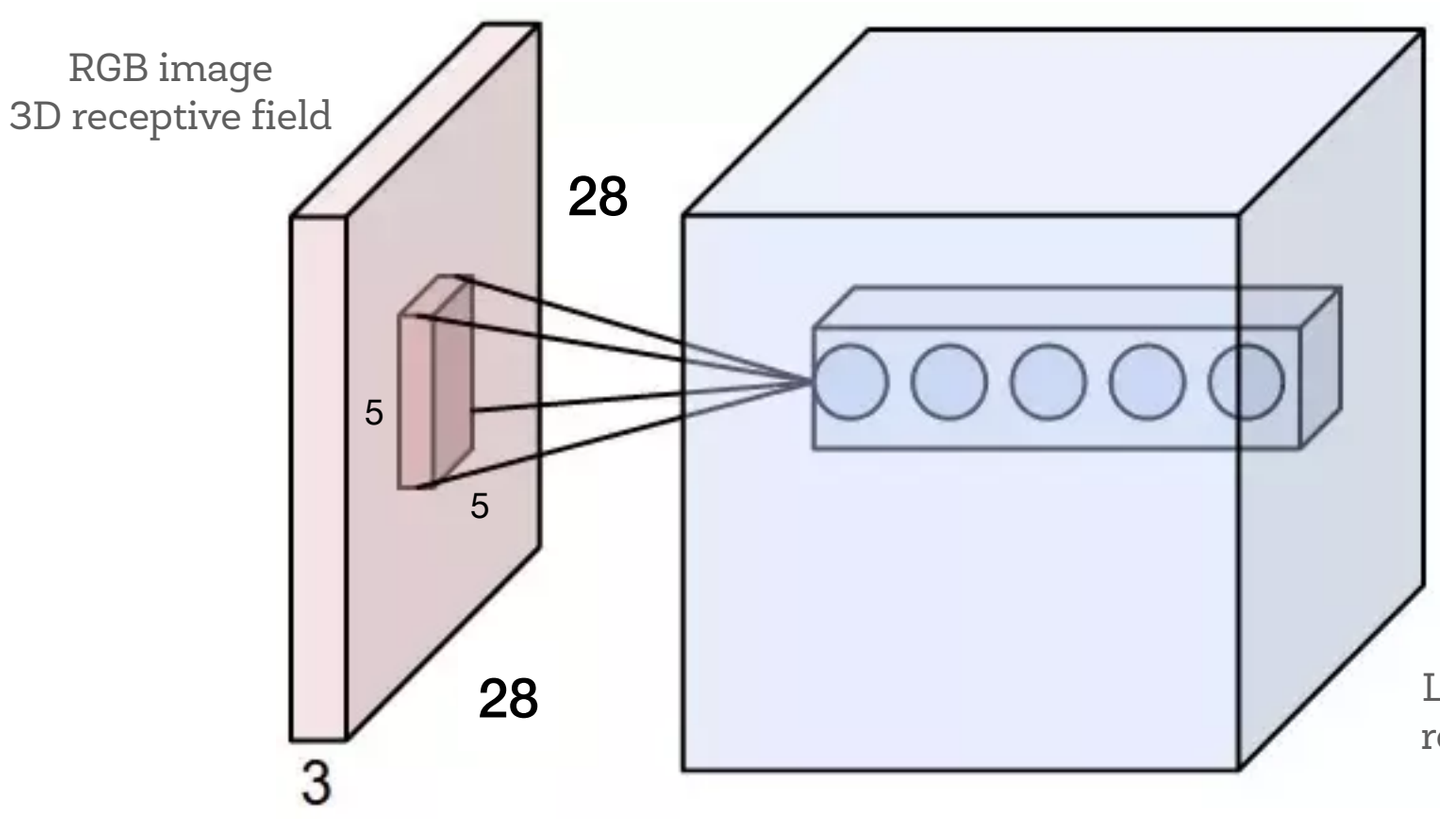
In fact, sometimes a different **stride** length is used. For instance, we might move the local receptive field 2 pixels to the right (or down), in which case we'd say a stride length of 2 is used.

This also **reduces ($\times 4$) the number of units of the first layer** and consequently the number of parameters.

http://neuralnetworksanddeeplearning.com/chap6.html

Minimizing the number of parameters: LRF's

To have more capacity, we could compute **several local receptive fields** per pixel and also to consider **3D local receptive fields** for color images:



Layer with multiple local receptive field responses (24 x 24 x 5)

This LRF layer has $\underbrace{25 \times 3}_{\text{parameters per receptive field}} \times \underbrace{5}_{\text{number of receptive fields per pixel}} \times \underbrace{24 \times 24}_{\text{number of units in the first layer}} + \underbrace{(24 \times 24 \times 5)}_{\text{biases}} = 218880$ parameters

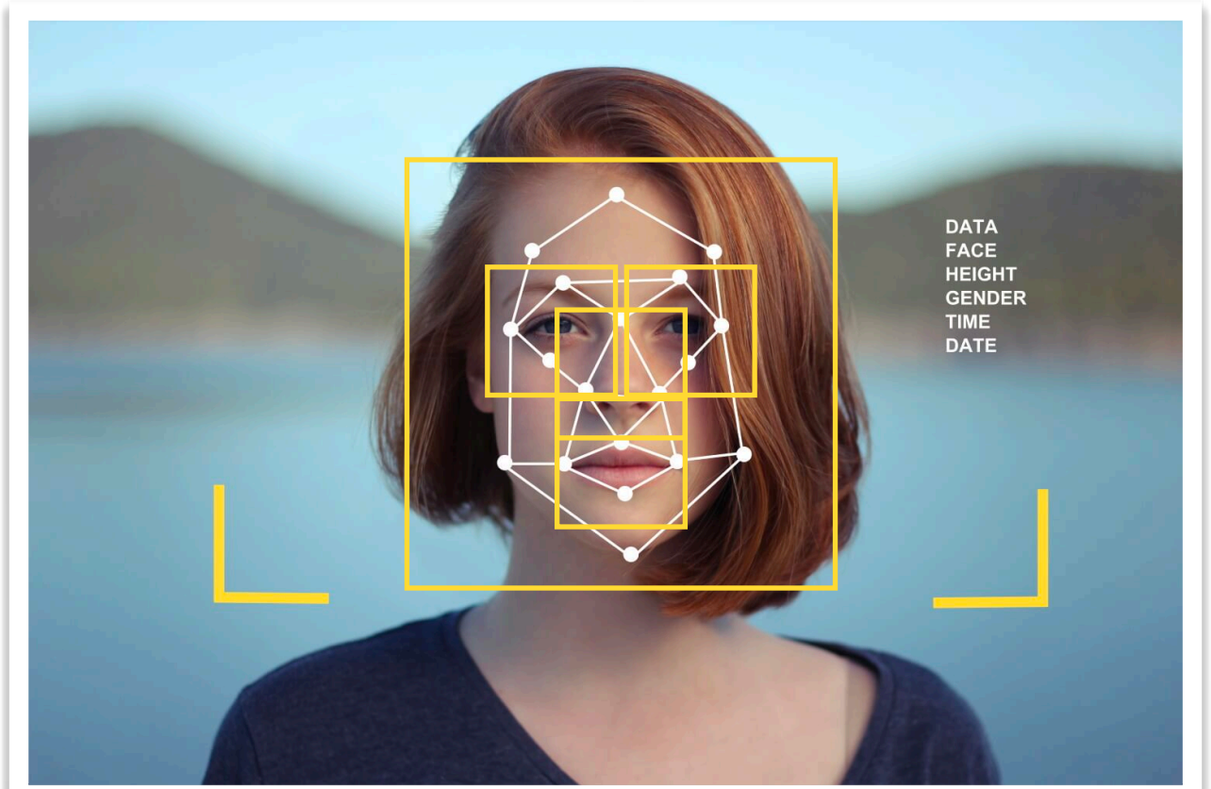
Too many!

From LRF's to Convolutions

Using LRF is like decomposing the problem of image analysis in a set of local experts that are specialized in certain parts of the image.

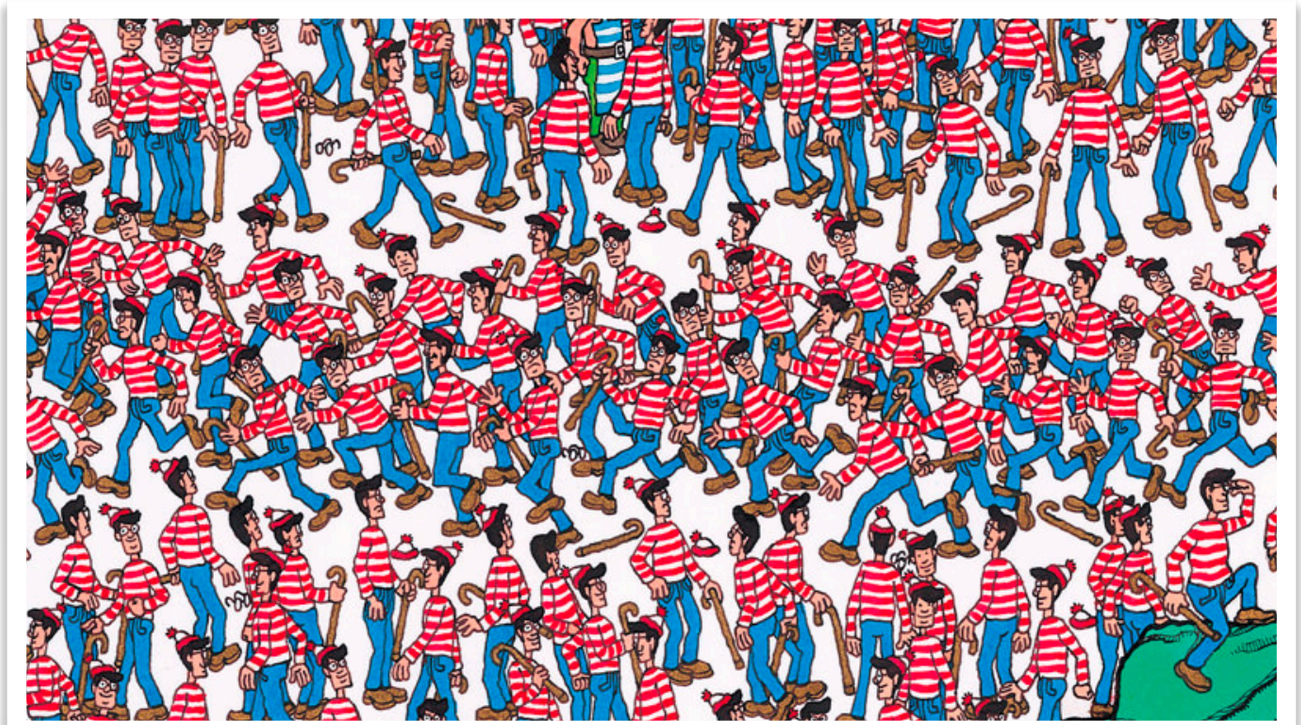
The outcome of these specialists can be integrated at a high level of the model by **stacking** layers (receptive fields over receptive fields).

This can be useful for some applications (where we dot not need translation-invariance): f.e. face recognition from detected faces.



From LRF's to Convolutions

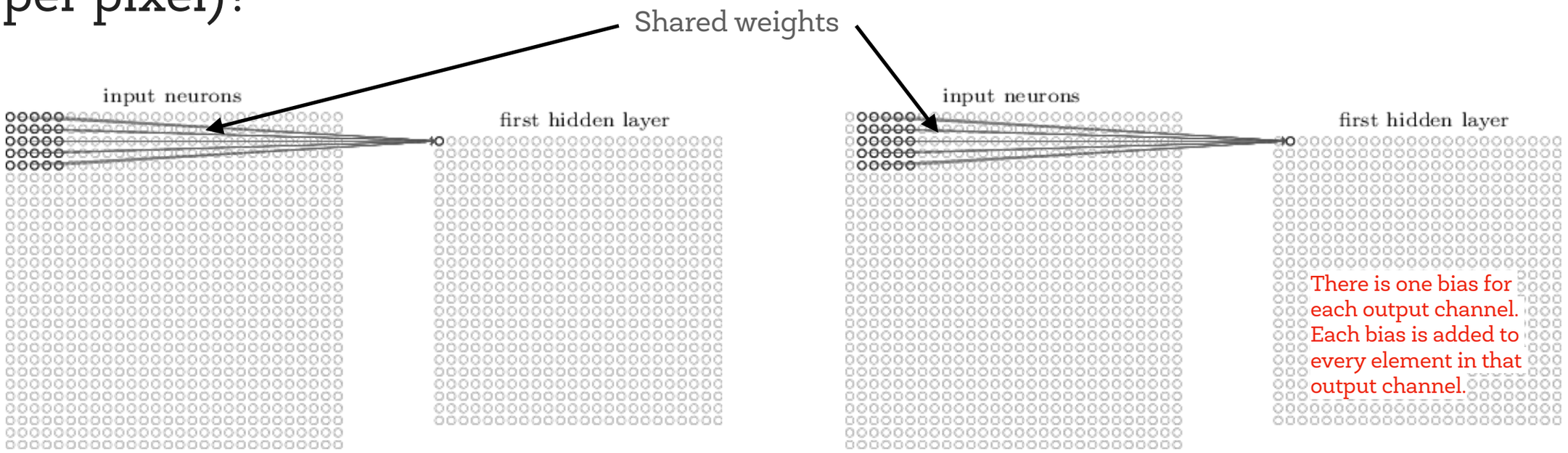
But in **natural images**, an object may appear at any image coordinates! Visual scene analysis must be invariant to translations in the image plane.



In this case **LRF are not suited for this task** because in order to learn a detector we should provide the model with a lot of examples in order to consider all possible object localizations!

Froom LRF's to convolutional layers

What about **sharing** LRF weights (and using several receptive fields per pixel)?



This can be expressed as a **convolution**. Now, for the (j, k) hidden unit, the output is of "shared" a receptive field can be expressed as:

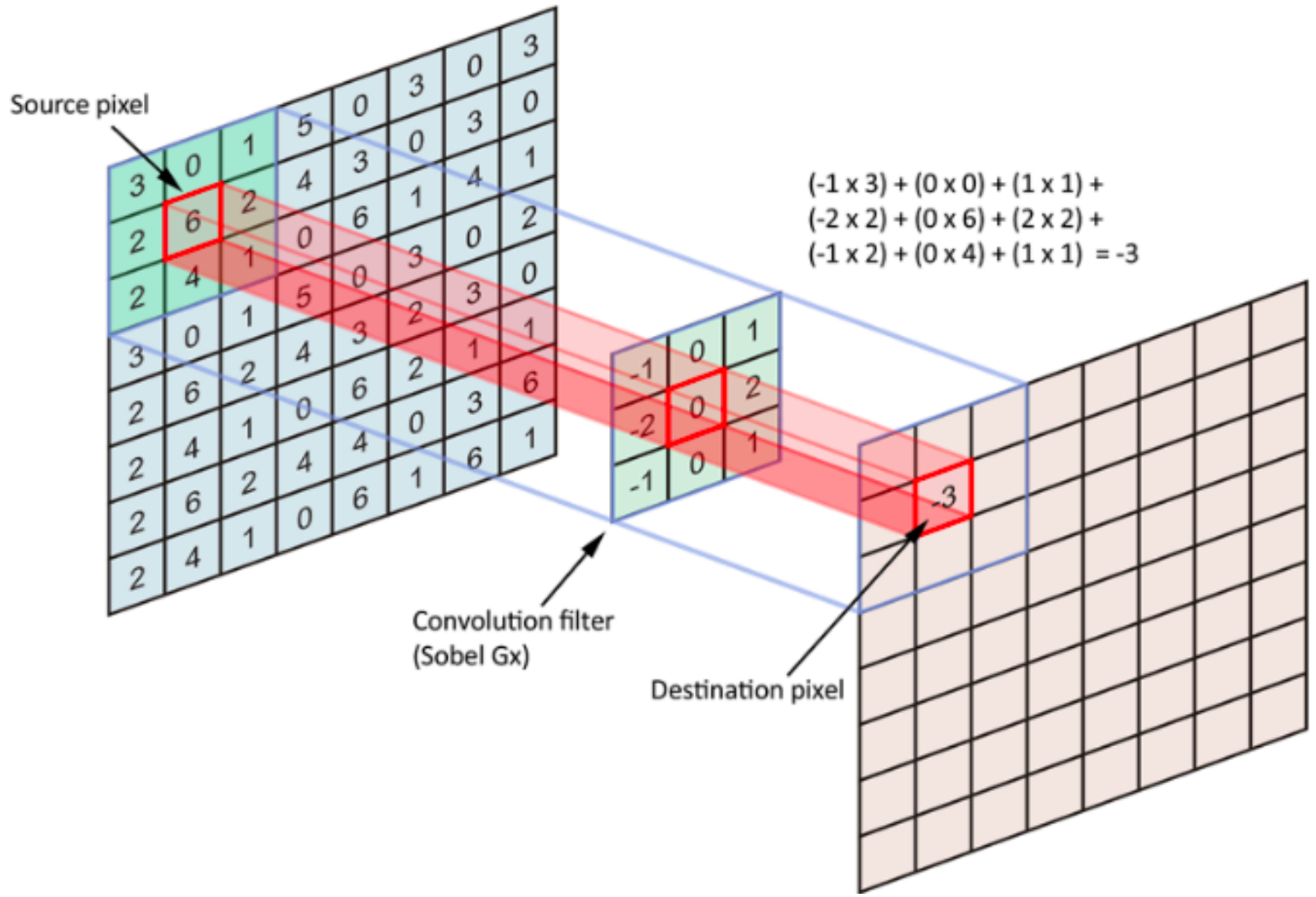
The non-linear activation function is necessary!

$$\sigma\left(b + \sum_{l=0}^{4}\sum_{m=0}^{4} w_{l,m} a_{j+l,k+m}\right)$$

where b is the shared value for the bias, $w_{l,m}$ is a 5x5 array of shared weights and $a_{x,y}$ represents the input value at position x, y .

Convolutional layers

The term **convolution** refers to the mathematical combination of two functions to produce a third function. It merges two sets of information.

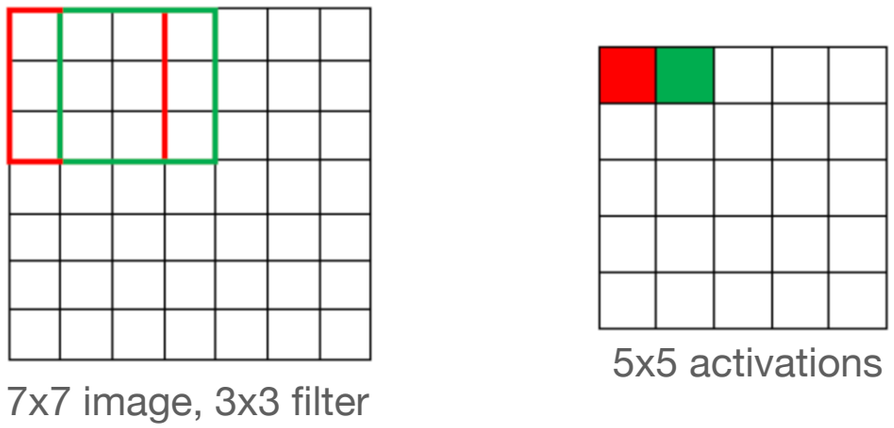


There is one bias for each output channel. Each bias is added to every element in that output channel.

https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2

Padding

For a gray scale (n x n) image and (f x f) filter/kernel, the dimensions of the image resulting from a convolution operation is (n – f + 1) x (n – f + 1).



Thus, the image shrinks every time a convolution operation is performed. This places an upper **limit to the number of times such an operation could be performed (due to stacking) before the image reduces to nothing** thereby precluding us from building deeper networks.

To overcome these problems, we use **padding**.

Padding

Padding is simply a process of adding “dimensions” to our input images so as to avoid the problems mentioned above.

Input: 6x6 image

0	0	0	0	0	0	0	0
0	1	2	3	1	3	5	0
0	2	2	5	4	2	5	0
0	0	6	9	6	2	2	0
0	2	0	1	9	4	0	0
0	5	5	4	6	7	6	0
0	6	1	3	7	1	5	0
0	0	0	0	0	0	0	0

0-padding

3x3 Filter

1	0	-1
1	0	-1
1	0	-1

*

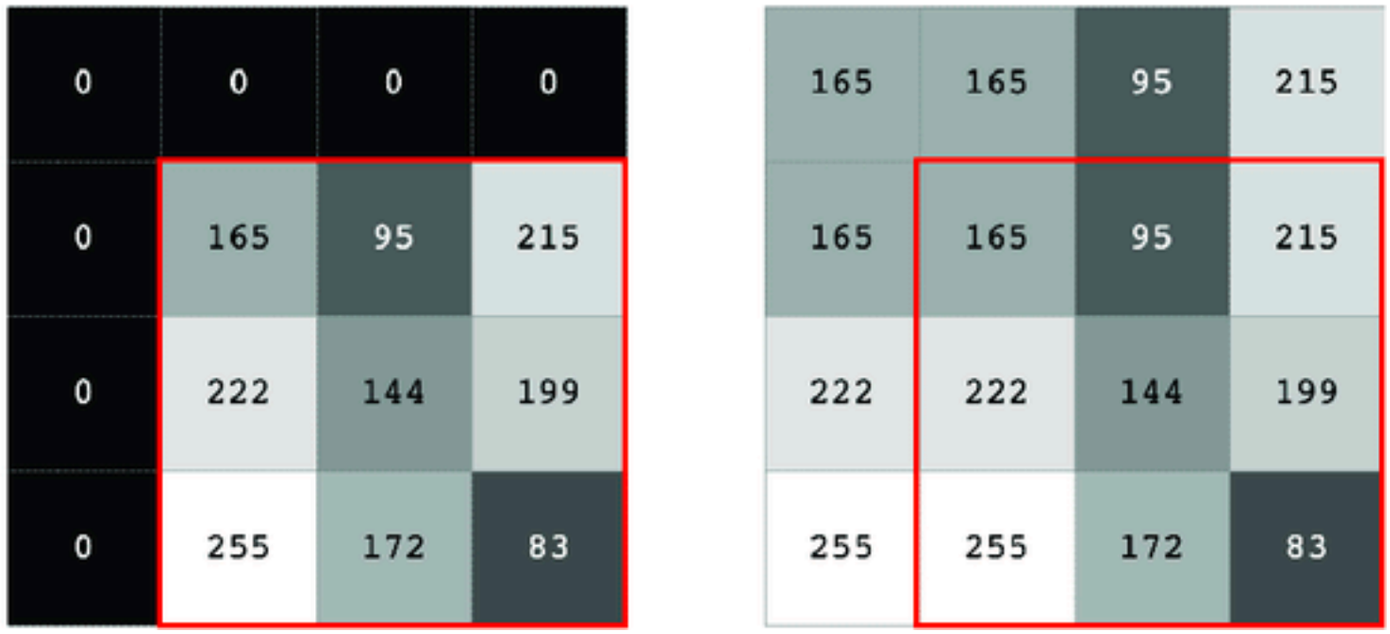
=

Output: 6x6 activations

-4	-5	-1	3	-5	5
-10	-14	-1	10	-1	7
-8	-11	-11	7	12	8
11	-7	-10	1	13	13
-6	5	-16	-4	10	12
-6	4	-7	-1	2	8

https://www.brilliantcode.net/1584/convolutional-neural-networks-1-convolution-layer-stride-padding-kernel/

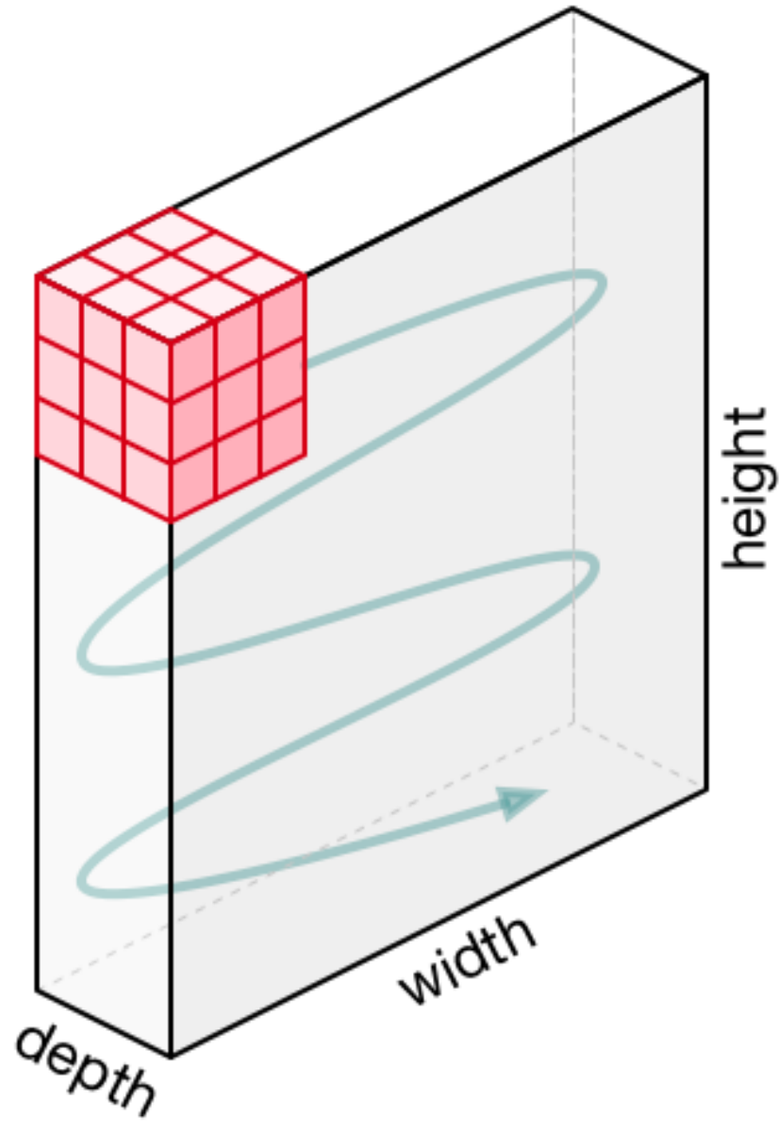
Padding



0-padding

Replication-padding

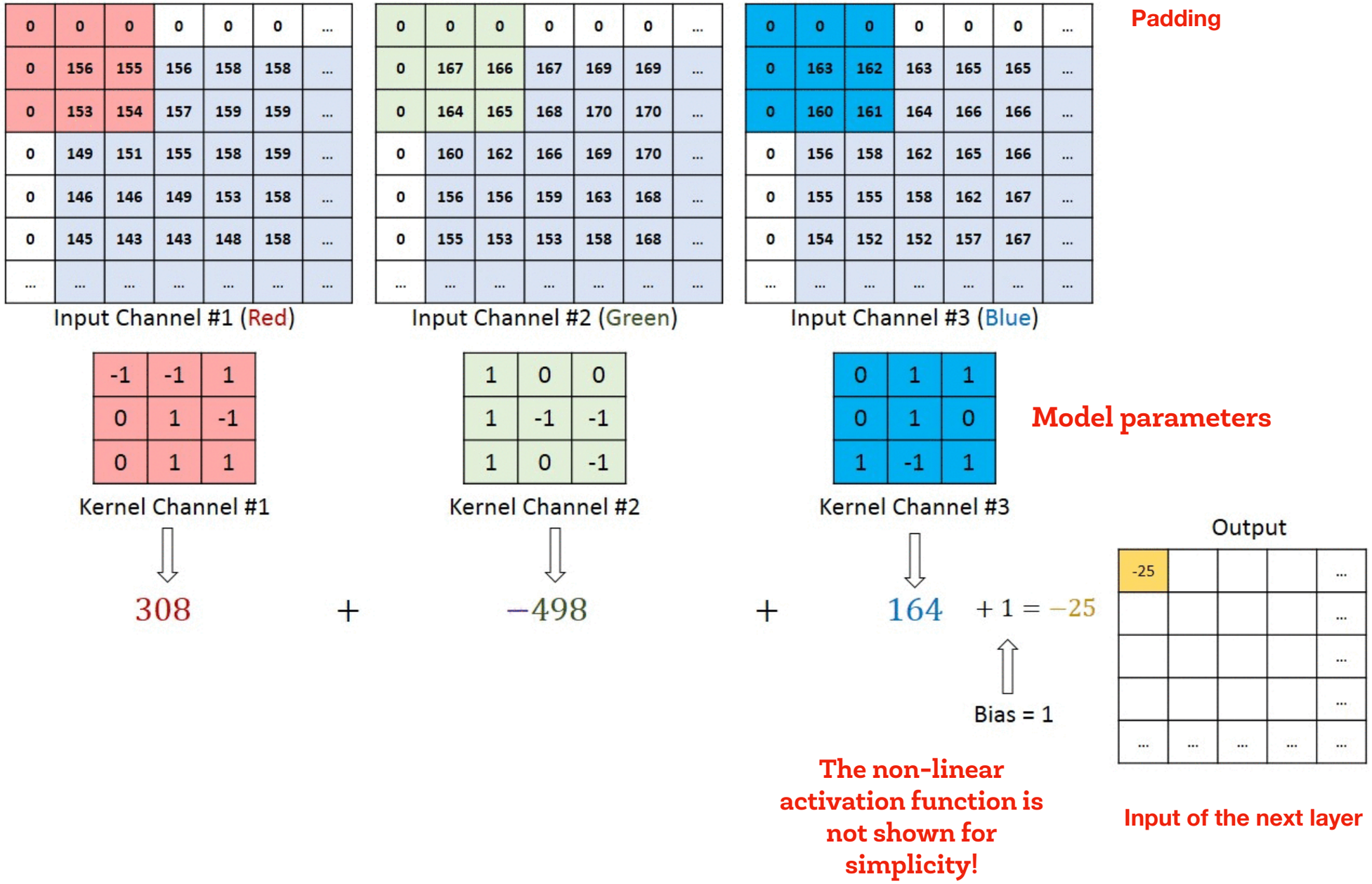
Convolutional layers (tensors)



https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Convolutional layers

From convolutions to convolutional layers: adding a non-linear activation function.



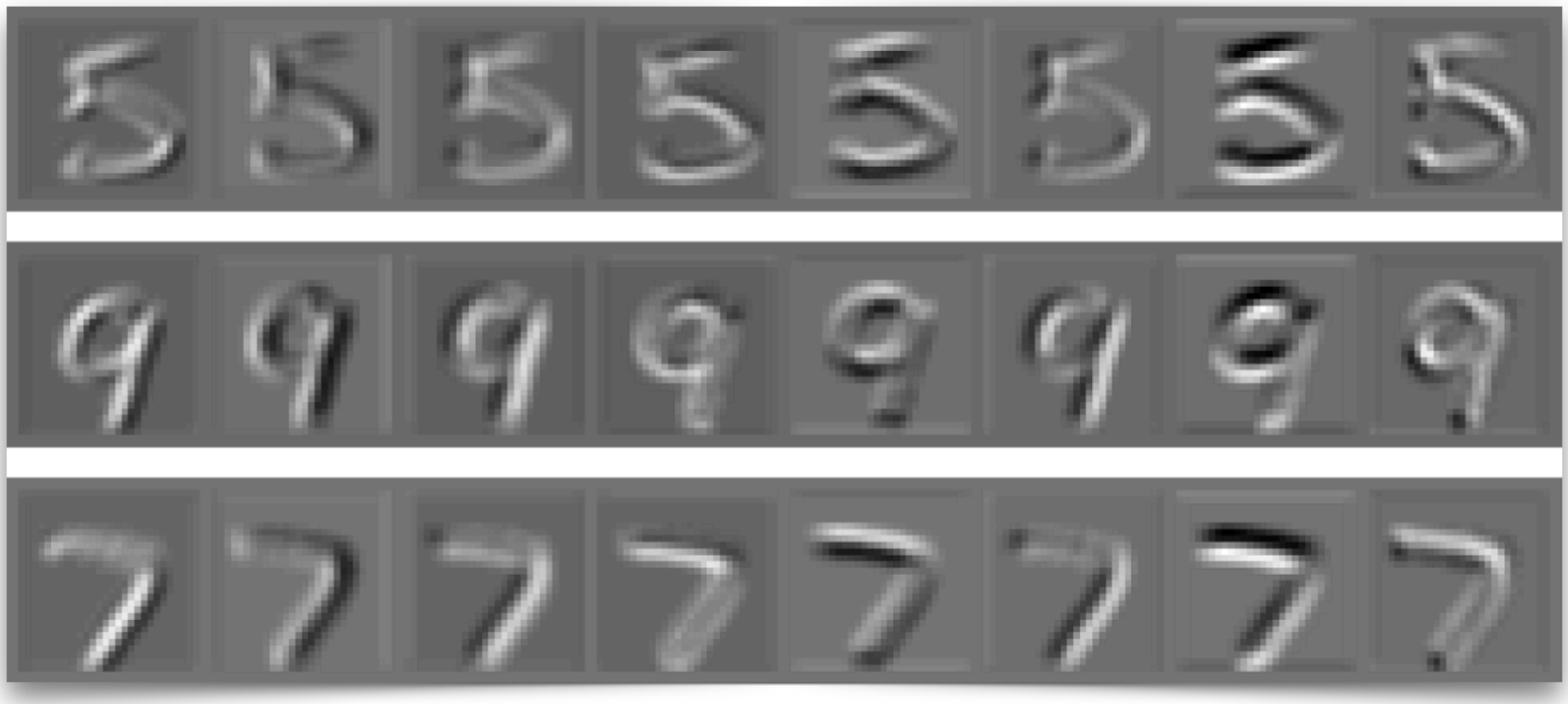
https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Convolutional layers

The weights of the first layer of a CNN can be visualized!
They are $n \times n \times 3$ (color images) or $n \times n \times 1$ (BW images) tensors.



Weights of conv1 layer, trained with BW MNIST, which has shape [8, 1, 5, 5]
(8 outer channels, 1 inner channel, 5x5 grids)

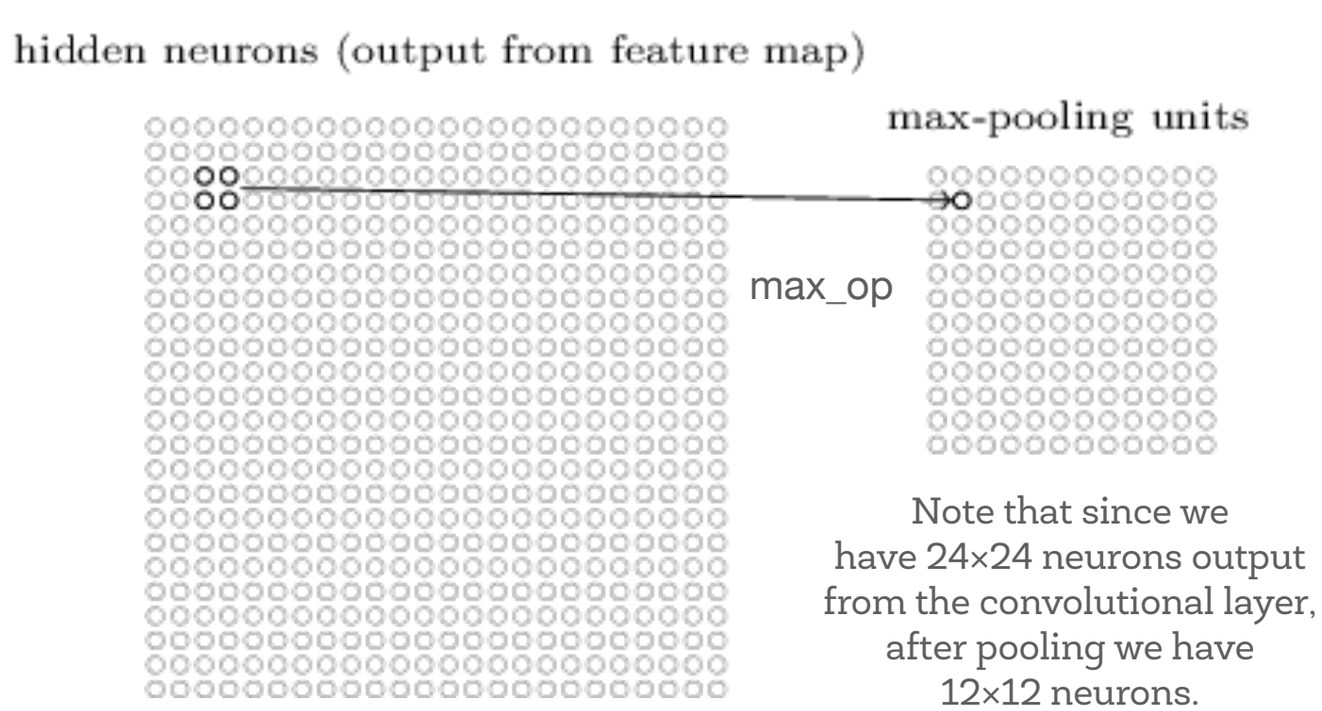


Filter output values on random inputs from MNIST.

Convolutional layers

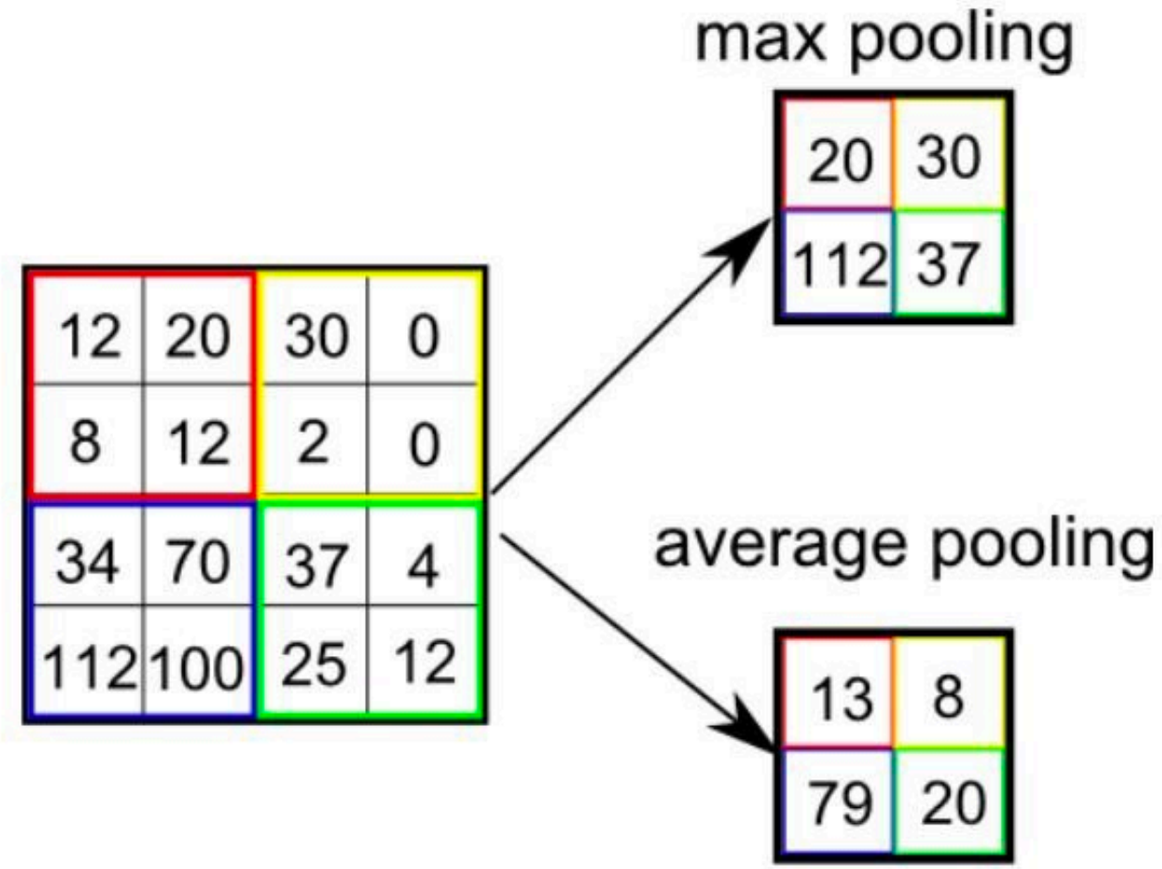
In addition to the convolutional layers just described, convolutional neural networks also contain **pooling layers**. Pooling layers are usually used immediately after convolutional layers. What the pooling layers do is **simplify the information in the output** from the convolutional layer.

As a concrete example, one common procedure for pooling is known as ***max-pooling***. In max-pooling, a pooling unit simply outputs the maximum activation in the 2×2 input region, as illustrated in the following diagram:



https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2

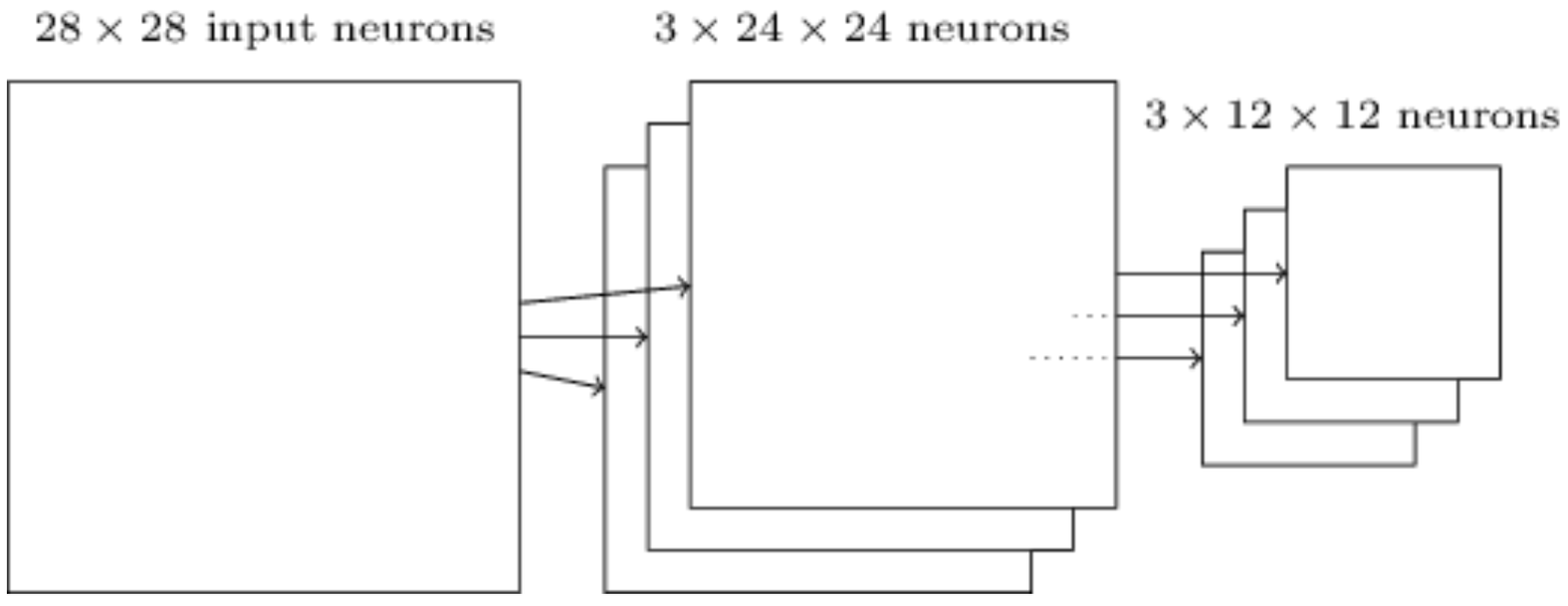
Convolutional layers



https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Convolutional layers

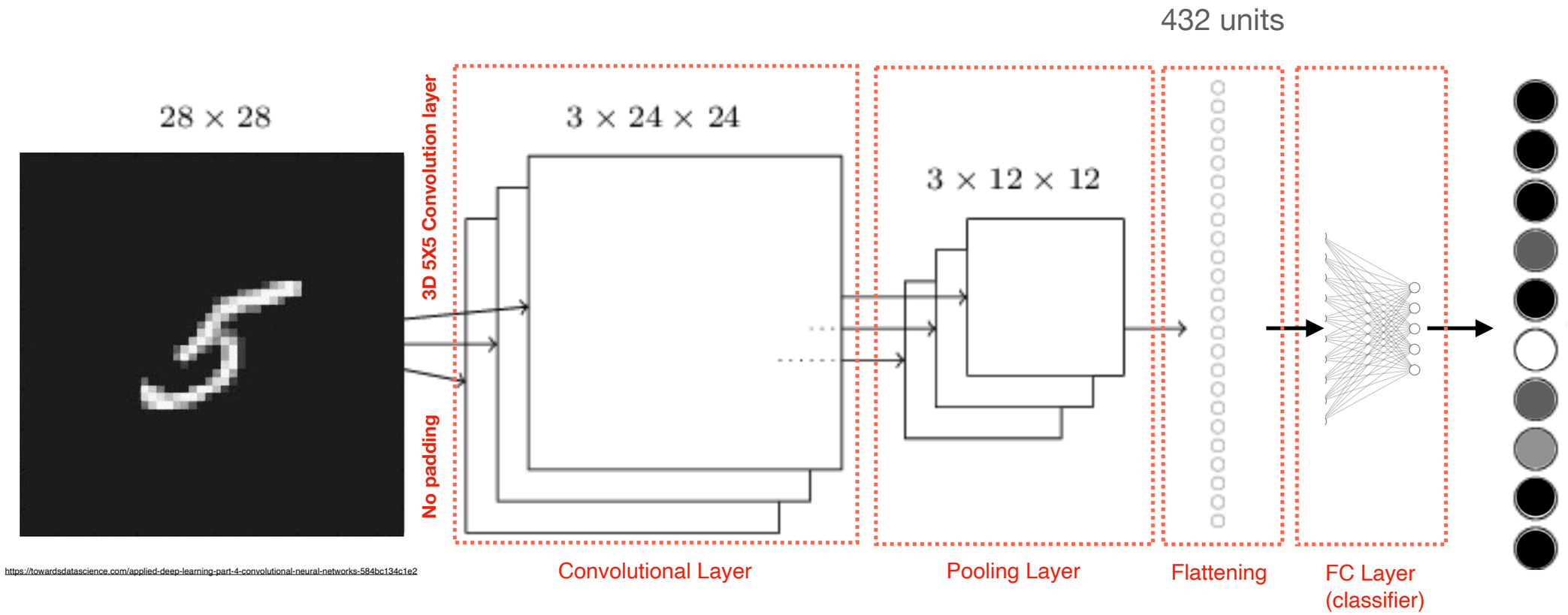
If there were 3 feature maps/convolution filters with no padding, the combined convolutional and max-pooling layers would look like:



https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2

Convolutional layers

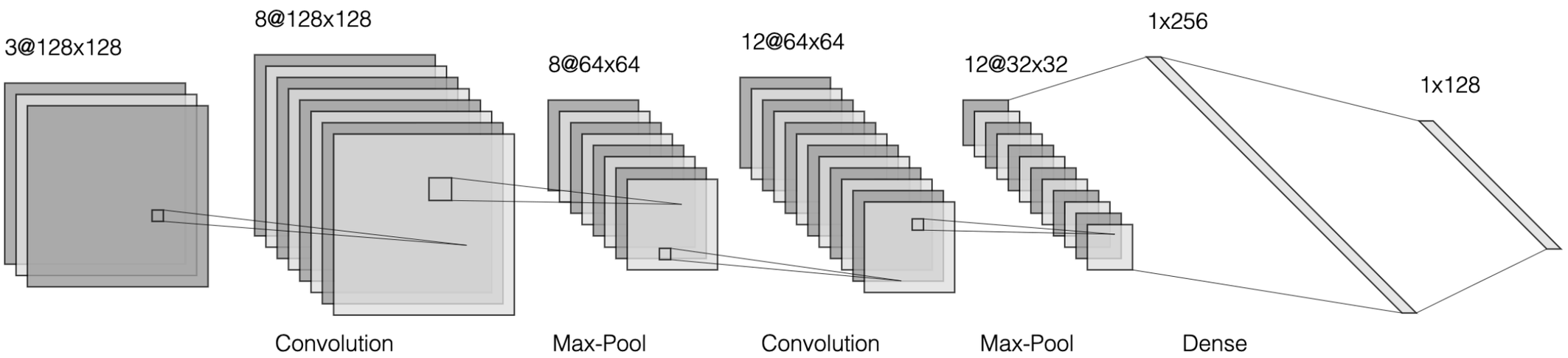
We can now put all these ideas together to form a complete **1-layer convolutional neural network**:



The network begins with 28×28 input neurons, which are used to encode the pixel intensities for the MNIST image. This is then followed by a convolutional layer using a 5×5 local receptive field and 3 feature maps. The result is a layer of 3×24×24 hidden feature neurons. The next step is a max-pooling layer, applied to 2×2 regions, across each of the 3 feature maps. The result is a layer of 3×12×12 hidden feature neurons. Finally, there is a fully connected layer with 3×12×12x10 + 10 parameters.

Convolutional layers and stacking

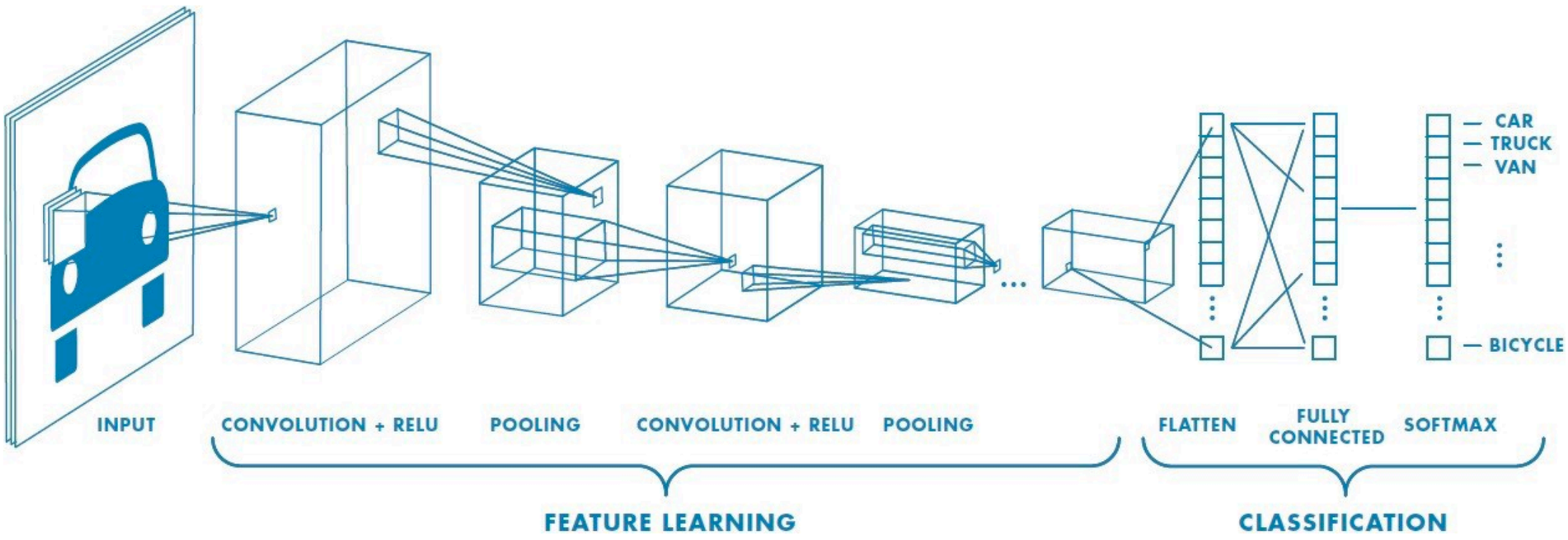
The **stacking** of convolutional layers allows a **hierarchical decomposition** of the input.



Convolutional layers and stacking

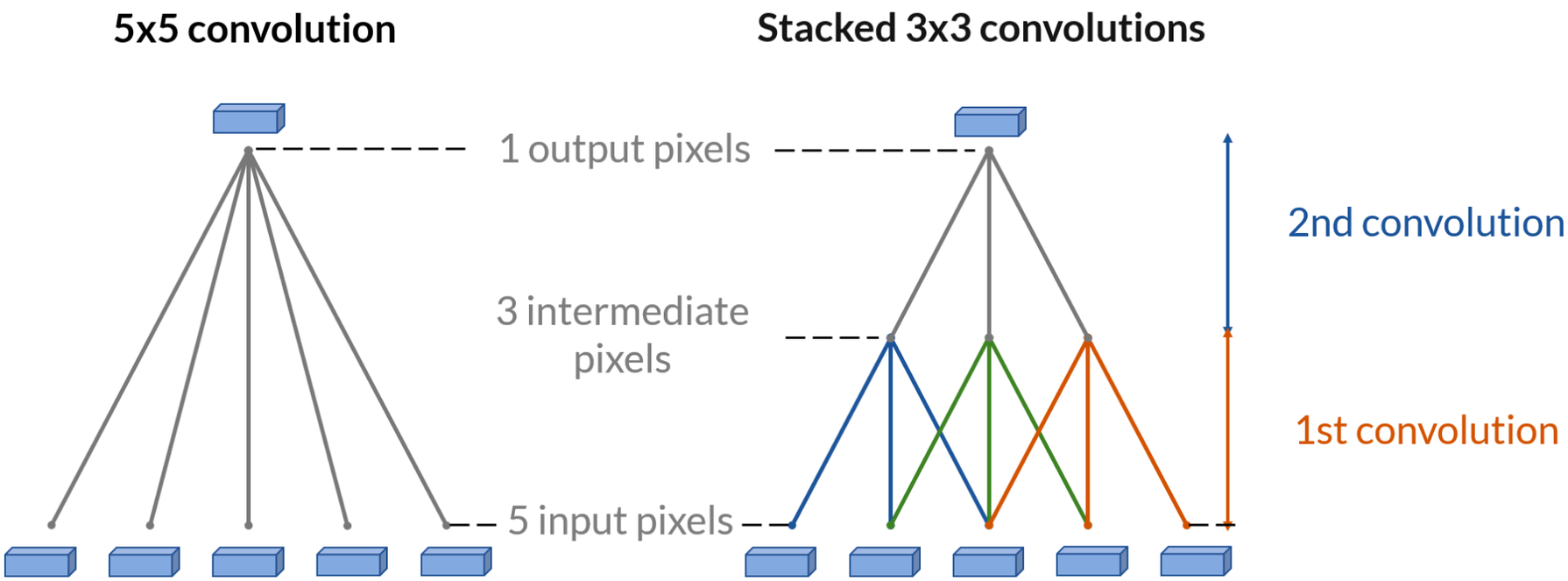
Consider that the filters that operate directly on the raw pixel values will learn to extract low-level features, such as lines and small patterns. The filters that operate on the output of the first line layers may extract features that are combinations of lower-level features, such as features that comprise multiple lines to express shapes.

This process continues until very deep layers are extracting faces, animals, houses, and so on.



https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Convolutional layers and stacking



We can replace large convolution kernels with multiple 3x3 convolutions on top of one another. This is good for two reasons: deeper is better and less computational cost.

Convolutional layers in Keras

```
1
2 model = keras.Sequential(
3     [
4         keras.Input(shape=input_shape),
5         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
6         layers.MaxPooling2D(pool_size=(2, 2)),
7         layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
8         layers.MaxPooling2D(pool_size=(2, 2)),
9         layers.Flatten(),
10        layers.Dropout(0.5),
11        layers.Dense(num_classes, activation="softmax"),
12    ]
13 )
14
15
16
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 32)        320
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 11, 11, 64)        18496
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 5, 5, 64)          0
_________________________________________________________________
flatten (Flatten)            (None, 1600)              0
_________________________________________________________________
dropout (Dropout)            (None, 1600)              0
_________________________________________________________________
dense (Dense)                (None, 10)                16010
=================================================================
Total params: 34,826
Trainable params: 34,826
Non-trainable params: 0
```

```
Test loss: 0.027600124478340 15
Test accuracy: 0.990000009536743 2
```

What about the inductive bias of a CNN?

```
# create model
model = kmod.Sequential()

# add model layers
model.add(klay.Conv2D(
    filters=32, kernel_size=3, activation='relu',
    input_shape=x_train.shape[1:],
    trainable=False))
model.add(klay.MaxPool2D(trainable=False))
model.add(klay.Conv2D(
    filters=32, kernel_size=3, activation='relu',
    kernel_initializer="glorot_uniform",
    trainable=False))
model.add(klay.MaxPool2D(trainable=False))
model.add(klay.Conv2D(
    filters=32, kernel_size=3, activation='relu',
    kernel_initializer="glorot_uniform",
    trainable=False))
model.add(klay.Flatten(trainable=False))
model.add(klay.Dense(
    units=10, activation='softmax',
    kernel_initializer="glorot_uniform"))
```

```
Model: "sequential_10"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_30 (Conv2D)           (None, 26, 26, 32)        320

max_pooling2d_20 (MaxPoolin  (None, 13, 13, 32)        0
g2D)

conv2d_31 (Conv2D)           (None, 11, 11, 32)        9248

max_pooling2d_21 (MaxPoolin  (None, 5, 5, 32)          0
g2D)

conv2d_32 (Conv2D)           (None, 3, 3, 32)          9248

flatten_10 (Flatten)         (None, 288)               0

dense_10 (Dense)             (None, 10)                2890

=================================================================
Total params: 21,706
Trainable params: 2,890
Non-trainable params: 18,816
```

```
Random Feature Accuracy = (91.49 +/- 0.58)%
```



What about the inductive bias of a CNN?

CNN architecture leverages some natural invariances (e.g. translations, small scale changes, small deformations) for free.

From Images to Computer Vision

We have seen how to implement image classification, but there are other Computer Vision tasks:

- Reconstruction (recover what is "not seen", f.e. 3D)
- Visual Control, tracking,...
- Segmentation
- Detection (bounding box)
- Etc.

CNNs can play an important role to solve this tasks, but we must reformulate them in terms of learning tasks (losses and architectures).